Varendra University

Dept. of Computer Science & Engineering (CSE)

Introduction to IBM PC Assembly Language (Chapter-4)

Course Title: **Microprocessor and Assembly Language**

Course No: **CSE 3205**

Prepared By:
Md. Adnan Sami
Lecturer, Dept. of CSE, VU

Outline

1. Assembly Language Syntax.
2. Name Field.
3. Operation Field.
4. Operand Field.
5. Comment Field.
6. Program Data.
7. Variable and Byte Variable.
8. Word Variable.
9. Arrays.
10. Named Constants.
11. A Few Basic Instructions.
12. Translation of High Level Language to Assembly Language

Outline

13. Memory Models.
14. Input and Output Instructions.
15. A Program for Displaying String.
16. A Program for Case Conversion.

Assembly Language Syntax

Assembly language programs are translated into machine language instructions by an assembler.

Statements:

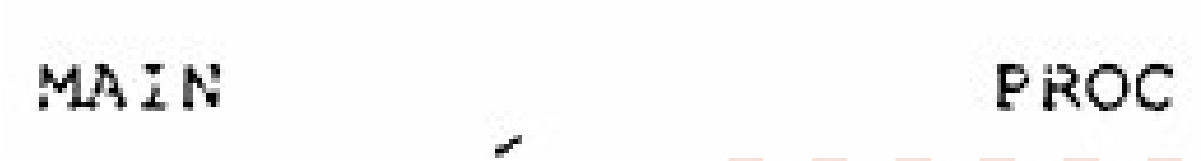
```
name      operation      operand(s)      comment

START:          MOV CX,5          ;initialize counter
```

Here,

Name field: START:
Operation: MOV
Operand: CX, 5
Comment: ;initialize counter

Assembly Language Syntax

Statements:



Here,

Name field: MAIN

Operation: PROC

Name Field

- The name field is used for instruction labels, procedure names, and variable names.
- The assembler does not differentiate between upper and lower case in a name.

Examples of legal names:

COUNTER1
@character
SUM_OF_DIGITS
$1000
DONE?
.TEST

Name Field

Examples of illegal names

`TWO WORDS`	contains a blank
`2abc`	begins with a digit
`A45.28`	. not first character
`YOU&ME`	contains an illegal character

Operation Field

Operation's function:

- MOV
- ADD
- SUB

Operand Field

Operand field specifies the data that are to be acted on by
the operation.

NOP		no operands, does nothing
INC	AX	one operand; adds 1 to the contents of AX
ADD	WORD1,2	two operands; adds 2 to the contents of memory word WORD1

Comment Field

- The comment field of a statement is used by the programmer to say something about what the statement does.
- A semicolon marks the beginning of this field, and the assembler ignores anything typed after the semicolon.
- Comments are optional but assembly language is so low-level, it is almost impossible to understand an

```
MOV CX,0                ;move .0 to CX
```

Instead, use comments to put the instruction into the context of the program:

```
MOV CX.0                ;CX counts terms, initially 0
```

Program Data

- The processor operates only on binary data. The assembler must translate all data representation into binary numbers.
- An assembly language program we may express data as binary, decimal, or hex numbers, and even as characters.

Program Data

Numbers:

- A binary number is written as a bit string followed by the letter “B" or "b“. **For example:** 1010B.
- A decimal number is a string of decimal digits, ending with an optional "D" or "d".
- A hex number must begin with a decimal digit and end with the letter "H" or "h“. **For example:** 0ABCH.

Program Data

Number	Type
11011	decimal
11011B	binary
64223	decimal
-21843D	decimal
1,234	illegal—contains a nondigit character
1B4DH	hex
1B4D	illegal hex number—doesn't end in "H"
FFFFH	illegal hex number—doesn't begin with a decimal digit
0FFFFH	hex

Program Data

Characters:

- Characters and character strings must be enclosed in single or double quotes. **For example:** "A" or 'hello'

Table 4.1 Data-Defining Pseudo-ops

Pseudo-op	*Stands for*
DB	define byte
DW	define word
DD	define doubleword (two consecutive words)
DQ	define quadword (four consecutive words)
DT	define tenbytes (ten consecutive bytes)

Variable and Byte Variable

Variable:

- Variables play the same role in assembly language that they do in high-level languages.
- Each variable has a data type and is assigned a memory address by the program.

Byte Variable

Byte variable takes the following form:

```
name   DB              initial_value
```

where the pseudo-op DB stands for "Define Byte".

For example,

```
ALPHA  DB              4
```

Variable and Byte Variable

Byte Variable

Byte variable is defined in the following form:

```
name   DB              initial_value
```

where the pseudo-op DB stands for "Define Byte".

For example,

```
ALPHA   DB              4
```

Here,

name: ALPHA.

initialial_value: 4.

Variable and Byte Variable

Byte Variable

A question mark (" ?") used in place of an initial value sets aside an uninitialized byte. For example:

```
BYT  DB          ?
```

Word Variable

Word variable is defined in the following form:

```
name   DW                initial_value
```

```
WRD    DW                 -2
```

Here,

name: WRD.

initialial_value: -2.

Arrays

In assembly language, an array is just a sequence of memory bytes or words.

For example:

To define a three-byte array called B_ARRAY, whose initial values are 10h, 20h, and 30h. We can write,

```
B_ARRAY        DB              10H,20H,30H
```

Arrays

The name B_ARRAY is associated with the first of these bytes, B_ARRAY+1 with the second, and B_ARRAY+2 with the third. If the assembler assigns the offset address 0200h to B_ARRAY, then memory would look like this:

Symbol	Address	Contents
B_ARRAY	200h	10h
B_ARRAY+1	201h	20h
B_ARRAY+2	202h	30h

Arrays

The name B_ARRAY is associated with the first of these bytes, B_ARRAY+1 with the second, and B_ARRAY+2 with the third. If the assembler assigns the offset address 0200h to B_ARRAY, then memory would look like

Symbol	Address	Contents
B_ARRAY	200h	10h
B_ARRAY+1	201h	20h
B_ARRAY+2	202h	30h

In the same way, an array of words may be defined. For exa

```
W_ARRAY     DW      1000,40,29887,329
```

Arrays

Character Strings:

```
LETTERS     DB      'ABC'

is equivalent to

LETTERS     DB      41H,42H,43H
```

Inside a string, the assembler differentiates between upper and lower case. Thus, the string "abc" is translated into three bytes with values 61h, 62h, and 63h.

Named Constants

EQU *(Equates)*

To assign a name to a constant, we can use the **EQU** (equates) pseudo-op. The syntax is

```
name            EQU             constant
```

For example, the statement

```
LF              EQU             0AH
```

assigns the name LF to 0Ah, the ASCII code of the line feed character. The name LF may now be used in place of 0Ah anywhere in the program. Thus, the assembler translates the instructions

```
MOV DL, 0AH
```

and

```
MOV DL, LF
```

into the same machine instruction.

The symbol on the right of an EQU can also be a string. For example,

```
PROMPT EQU 'TYPE YOUR NAME'
```

Then instead of

```
MSG DB      'TYPE YOUR NAME'
```

we could say

```
MSG DB          PROMPT
```

Note: no memory is allocated for EQU names.

A Few Basic Instructions

MOV and XCHG:

The MOV Instruction is used to transfer data between registers, between a register and a memory location, or to move a number directly into a register or memory location. The syntax is,

```
MOV   destination,source
```

Example:

```
MOV   AX,WORD1
```

The contents of register AX are replaced by the contents of memory location WORD1. The contents of WORD1 are unchanged.

A Few Basic Instructions

MOV and XCHG:

MOV AX, BX
AX gets what was previously in BX. But BX is unchanged.

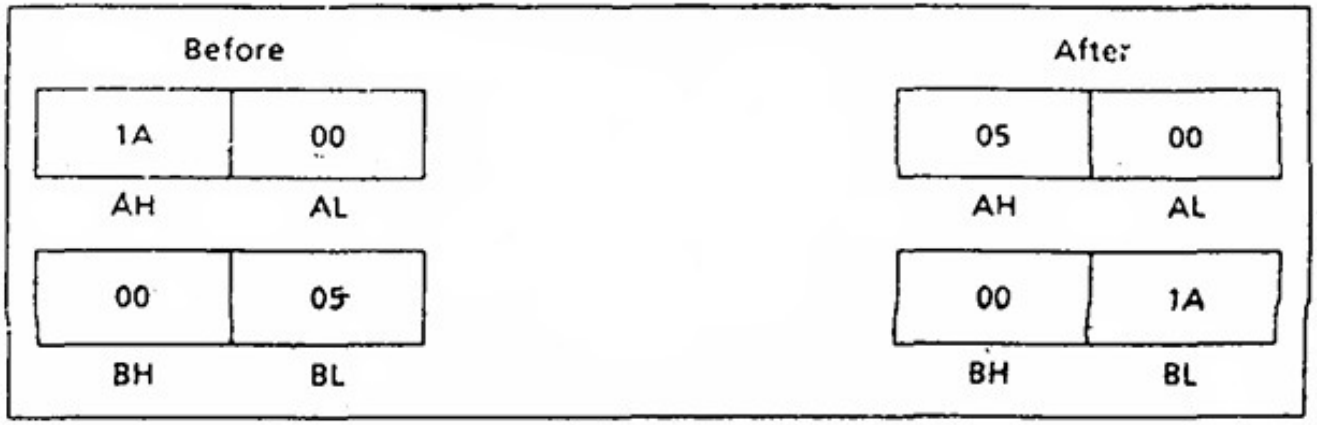


Figure 4.2 XCHG AH,BL

A Few Basic Instructions

MOV and XCHG:

Table 4.2 Legal Combinations of Operands for MOV and XCHG

MOV

	Destination Operand			
Source Operand	*General register*	*Segment register*	*Memory location*	*Constant*
General register	yes	yes	yes	no
Segment register	yes	no	yes	no
Memory location	yes	yes	no	no
Constant	yes	no	yes	no

XCHG

	Destination Operand	
Source Operand	*General register*	*Memory location*
General register	yes	yes
Memory location	yes	no

A Few Basic Instructions

MOV and XCHG:

XCHG (exchange) operation is used to exchange the contents
of two registers, or a register and a memory location.
The syntax i

```
XCHG   destination,source
```

An example is

```
XCHG   AH,BL
```

This instruction swaps the contents of AH and BL, so that AH contains what was previously in BL and BL contains what was originally in AH.

A Few Basic Instructions

MOV and XCHG:

Another example is,

```
XCHG   AX,WORD1
```

Swaps the contents of AX and memory location WORD1.

A Few Basic Instructions

Restrictions on MOV and XCHG:

There are a few restrictions on the use of MOV and XCHG.

MOV or XCHG between memory locations is not allowed. Fo

```
ILLEGAL: MOV  WORD1,WORD2
```

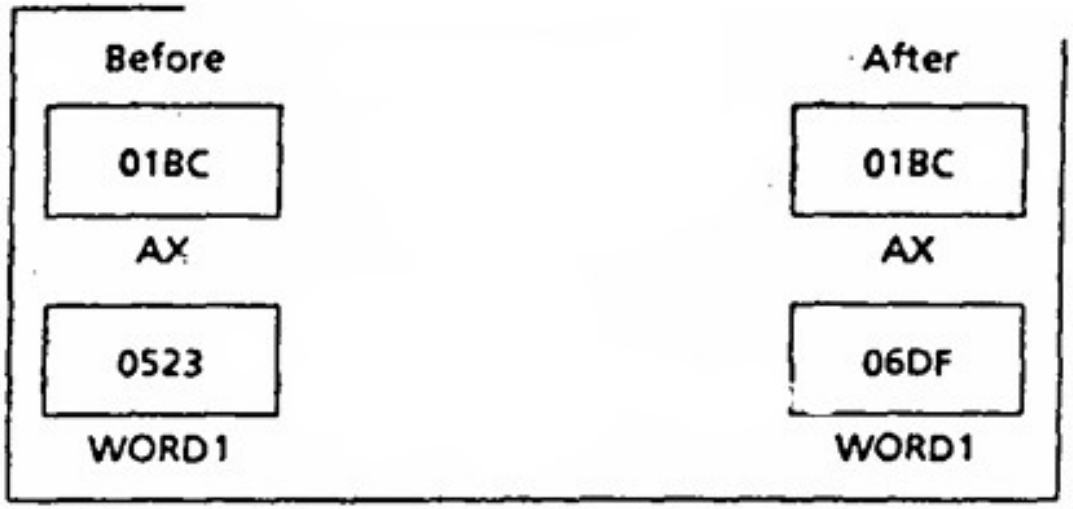
Solution:

We can solve th ng a register.

```
MOV   AX,WORD2
MOV   WORD1,AX
```

A Few Basic Instructions

Figure 4.3 ADD WORD1,AX



A Few Basic Instructions

ADD, SUB, INC, DEC:

ADD and SUB instructions are used to add or subtract the contents of two registers, a register and a memory location. The syntax is,

```
ADD   destination,source
SUB   destination,source
```

Example:

i)

```
ADD   WORD1,AX
```

This instruction, **"Add AX to WORD1"** causes the contents of AX and memory word WORD1 to be added, and the sum is stored in WORD1. AX is unchanged.

A Few Basic Instructions

ADD, SUB, INC, DEC:

Example:

ii)

```
SUB   AX,DX
```

This instruction, **"Subtract DX from AX"** the value of DX is subtracted from the value of AX, with the difference being stored in AX. DX is unchanged.

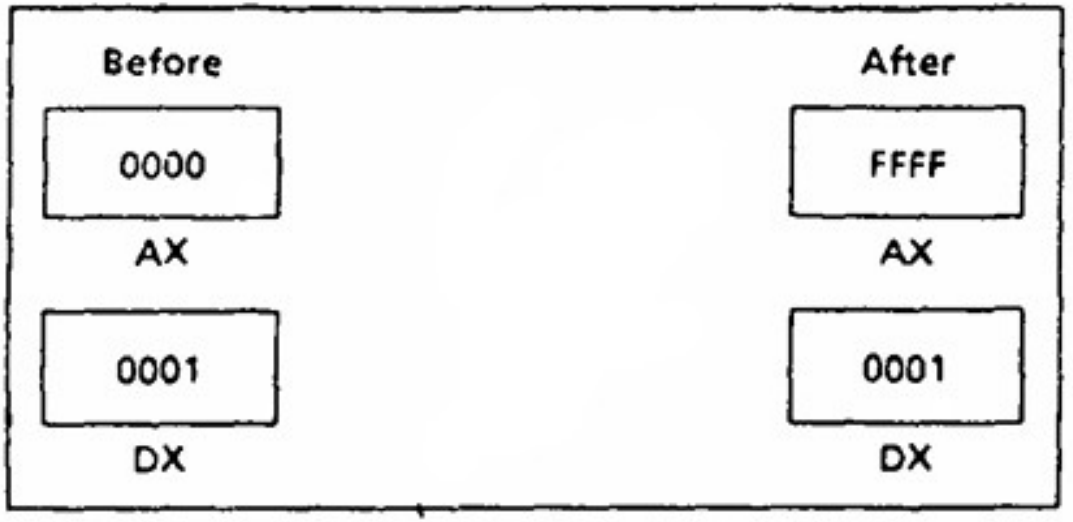
Table 4.3 Legal Combinations of Operands for ADD and SUB

	Destination Operand	
Source Operand	*General register*	*Memory location*
General register	yes	yes
Memory location	yes	no
Constant	yes	yes

A Few Basic Instructions

ADD, SUB, INC, DEC:

Figure 4.4 SUB AX,DX



A Few Basic Instructions

ADD, SUB, INC, DEC:

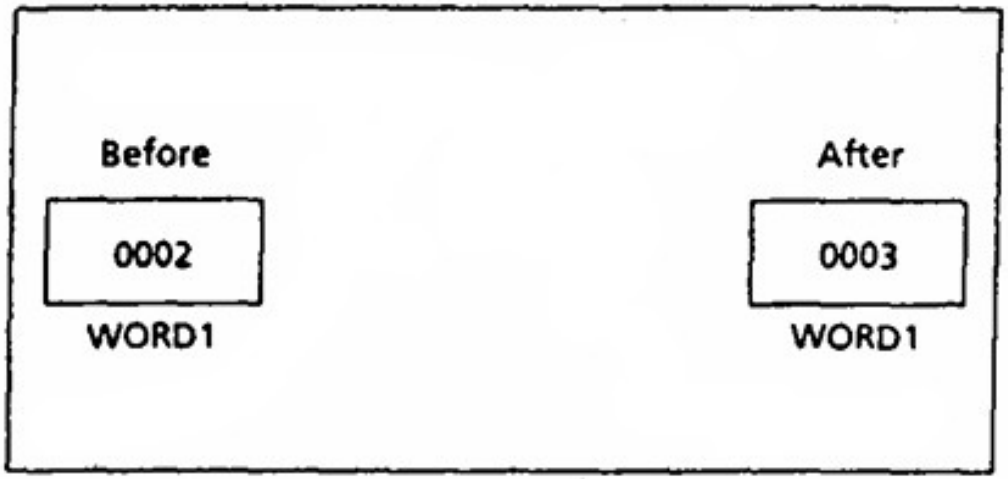
INC (increment) is used to add 1 to the contents of a register or memory location. The syntax is,

```
INC destination
```

Example:

```
INC WORD1

adds 1 to the contents of WORD1
```

Figure 4.5 INC WORD1



A Few Basic Instructions

ADD, SUB, INC, DEC:

DEC (decrement) is used to subtract 1 from a register or memory location. The syntax is,

```
DEC destination
```

Example:

```
DEC BYTE1
```

subtracts 1 from variable BYTE1

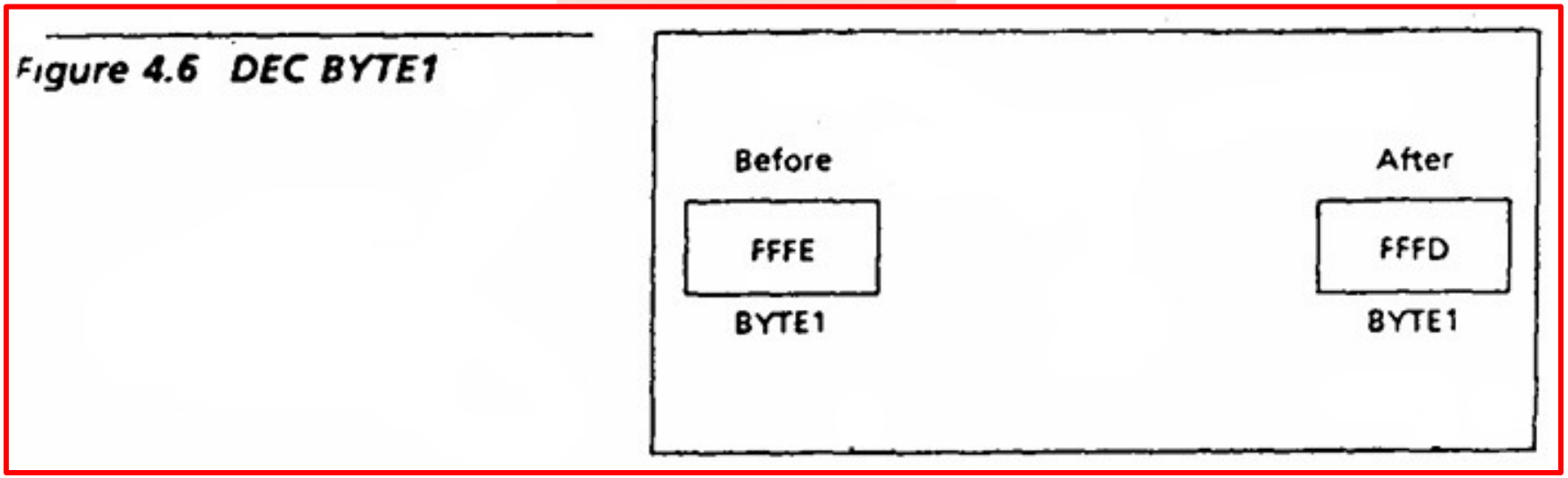


Figure 4.6 DEC BYTE1

A Few Basic Instructions

ADD, SUB, INC, DEC:

NEG is used to negate the contents of the destination. NEG does this by replacing the contents by its two's complement. The syntax is,

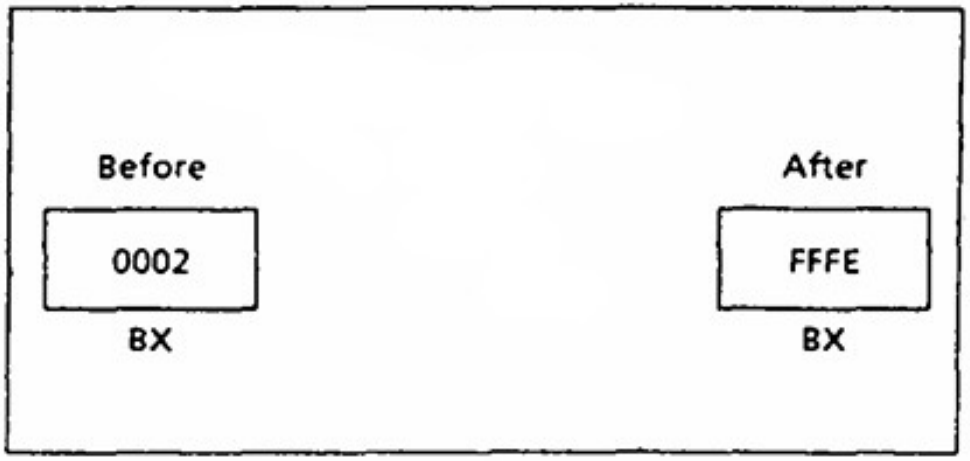
```
NEG  destination
```

Example:

```
NEG BX
```

Negates the contents of BX.

Figure 4.7 NEG BX



Translation of High Level Language to Assembly Language

Statement	*Translation*	
B = A	MOV AX, A	;move A into AX
	MOV B, AX	;and then into B

As was pointed out earlier, a direct memory–memory move is illegal, so we must move the contents of A into a register before moving it to B.

Translation of High Level Language to Assembly Language

```
A = 5 - A        MOV  AX, 5        ;put 5 in AX
                 SUB  AX, A        ;AX contains 5 - A
                 MOV  A, AX        ;put it in A
```

This example illustrates one approach to translating assignment statements: do the arithmetic in a register—for example, AX—then move the result into the destination variable. In this case, there is another, shorter way:

```
                 NEG  A.           ;A = -A
                 ADD  A, 5         ;A = 5 - A
```

The next example shows how to do multiplication by a constant.

```
A = B - 2 x A    MOV  AX, B        ;AX has B
                 SUB  AX, A        ;AX has B - A
                 SUB  AX, A        ;AX has B - 2 x A
                 MOV  A, AX        ;move result to A
```

Memory Models

The size of code and data a program can have is determined by specifying a memory model using the .MODEL directive. The syntax is,

```
.MODEL                    memory_model
```

Table 4.4 Memory Models

Model	*Description*
SMALL	code in one segment data in one segment
MEDIUM	code in more than one segment data in one segment
COMPACT	code in one segment data in more than one segment
LARGE	code in more than one segment data in more than one segment no array larger than 64k bytes
HUGE	code in more than one segment data in more than one segment arrays may be larger than 64k bytes

Input and Output Instructions

There are two ways to do input and output on the IBM PC:

1) By direct communication with I/O devices.
2) By using BIOS or DOS routines.

BIOS (Basic Input Output System) routine: Stored in ROM and interact directly with the I/O ports.
DOS routine: Carry out more complex tasks. For example: Printing a character string.

CPU communicates with the peripherals through I/O registers called I/O ports.

Input and Output Instructions

Function number	Routine
1	single-key input
2	single-character output
9	character string output

A Program for Displaying String

```
.model small
.stack 100h

.data
a db 'Dept. of CSE $';Variable declaration

.code

main proc

    mov ax,@data ;Initialize the data segment to code segment
    mov ds,ax

    mov ah,9
    lea dx,a ;Load Effective Address
    int 21h

    exit:
    mov ah,4ch
    int 21h
    main endp
end main
```

A Program for Case Conversion

Lowercase to Uppercase:

```
.model small
.stack 100h
.code

main proc
    mov ah,1
    int 21h     ;input
    mov bl,al

    sub bl,32

    mov ah,2
    mov dl,10
    int 21h
    mov dl,13
    int 21h

    mov ah,2
    mov dl,bl
    int 21h

    exit:
    mov ah,4ch
    int 21h
    main endp
end main
```

A Program for Case Conversion

Uppercase case to Lowercase:


```
.model small
.stack 100h
.code

main proc
    mov ah,1
    int 21h     ;input
    mov bl,al

    add bl,32

    mov ah,2
    mov dl,10
    int 21h
    mov dl,13
    int 21h

    mov ah,2
    mov dl,bl
    int 21h

    exit:
    mov ah,4ch
    int 21h
    main endp
end main
```